PROJECT REPORT

ABSTRACT

In the contemporary cybersecurity environment, conventional defenses like firewalls and antivirus programs frequently neglect the importance of the Domain Name System (DNS). DNS is critical for virtually all online activities, rendering it a prime target for cyber attackers who exploit its inherent security vulnerabilities. The insecure nature of DNS traffic, primarily characterized by its lack of encryption and authentication, opens avenues for cache poisoning attacks. These attacks can lead to severe consequences, including service disruptions, redirections, and data theft.

To address these vulnerabilities, the Domain Cache Detector has been developed as a web application built on the Django framework. This tool is designed to detect DNS cache poisoning in real time. It utilizes DNSSEC validation with the dnspython library and incorporates a Random Forest machine learning model to analyze and classify DNS responses. Key features such as Time to Live (TTL), IP diversity, and response latency are extracted for this analysis. The findings from these assessments are presented on an interactive dashboard, thereby improving threat visibility and bolstering the overall security of DNS.

TABLE OF CONTENTS

ABSTRACT 2
CHAPTER 1: INTRODUCTION 4
1.1 How does DNS Caching Work? 5
1.2 How does DNS caching work ? 7
1.3 Problem Statement 8
1.4 Objectives 8
CHAPTER 2: SYSTEM DESIGN AND METHODOLOGY 9
2.1 DNS Query 9
2.2 Feature Extraction 9
2.3 AI Model 10
2.4 Threat Level 10
CHAPTER 3: LITERATURE REVIEW 11
3.1 Historical Background 11
3.2 DNS Cache Poisoning Research 11
CHAPTER 4: SYSTEM DESIGN 13
4.1 Architecture Overview 13
4.2 Algorithms Used 14
CHAPTER 5 IMPLEMENTATION DETAILS 15
5.1 “dnspython” Integration 15
5.2 Django Integration 15
5.3 Machine Learning Model Training 15
5.4 Dashboard Interface 15
5.5 Workflow Integration 16
CHAPTER 6 SECURITY AND PERFORMANCE ANALYSIS 15
6.1 Security Perspective 17
6.2 Performance Perspective ..... **Error! Bookmark not defined.**
CHAPTER 7: RESULTS / SCREENSHOTS 18
Conclusion and Future Work 20

List of Tables

Table 1 : Comparative Studies.....12

Table 2 : Technology Stack Overview 13

Table 3 : Advantages over Traditional Methods 17

List of Figures

Figure 1: DNS Uncached Response 6

Figure 2 : DNS Cached Response 6

Figure 3 : DNS Cache Poisoning Process 7

Figure 4 : Poisoned DNS cache 7

Figure 5 : Work Flow Diagram..... 9

Figure 6 : Dashboard.....18

Figure 7 : DNS Threat Monitoring Dashboard – Recent Events Table 18

Figure 8 : Live DNS Threat Alerts Panel 19

CHAPTER 1: INTRODUCTION

The Domain Name System (DNS) functions as the essential directory of the Internet, converting user-friendly domain names, such as example.com, into numerical IP addresses that computers utilize for mutual identification and communication. Without the DNS, users would face the challenge of memorizing complex numerical addresses to navigate the web effectively.

Despite its critical role in Internet functionality, the DNS was not established with robust security features, making it susceptible to considerable threats. A major vulnerability is DNS cache poisoning, commonly referred to as DNS spoofing. In such scenarios, malicious attackers can inject misleading data into a DNS resolver's cache, resulting in users being redirected to fraudulent or harmful websites. This deceptive redirection can lead to severe consequences, including data theft, phishing attacks, and service interruptions. A fundamental aspect of this vulnerability is that DNS resolvers typically trust incoming responses without adequate verification, allowing poisoned entries to remain active until the cache naturally expires.

To address these security challenges, the Domain Name System Security Extensions (DNSSEC) were developed to enhance DNS query security through cryptographic protections. However, despite the existence of DNSSEC, its implementation remains limited, leaving numerous systems vulnerable to attack. To effectively combat these security issues, there is an urgent need for the development of more intelligent and autoted detection methodologies that can identify DNS threats in real-time.

1.1 How does DNS Caching Work?

A DNS resolver will save responses to IP address queries for a certain amount of time. In this way, the resolver can respond to future queries much more quickly, without needing to communicate with the many servers involved in the typical DNS resolution process. DNS resolvers save responses in their cache for as long as the designated time to live (TTL) associated with that IP address allows them to.

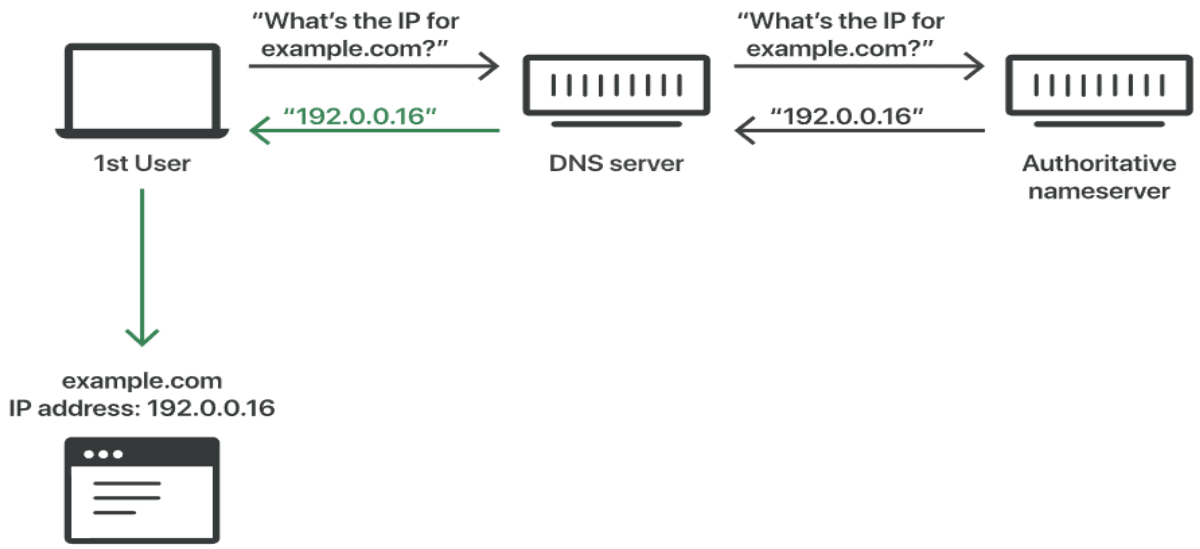

Figure 1: DNS Uncached Response

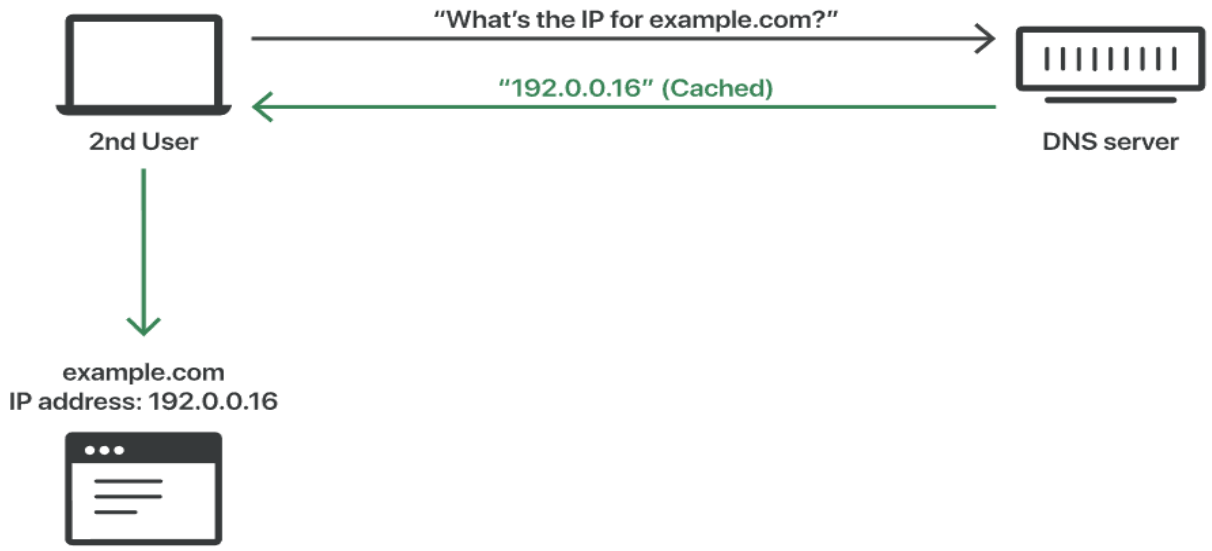

Figure 2 : DNS Cached Response

1.2 How does DNS caching work ?

A DNS resolver will save responses to IP address queries for a certain amount of time. In this way, the resolver can respond to future queries much more quickly, without needing to communicate with the many servers involved in the typical DNS resolution process. DNS resolvers save responses in their cache for as long as the designated time to live (TTL) associated with that IP address allows them to.

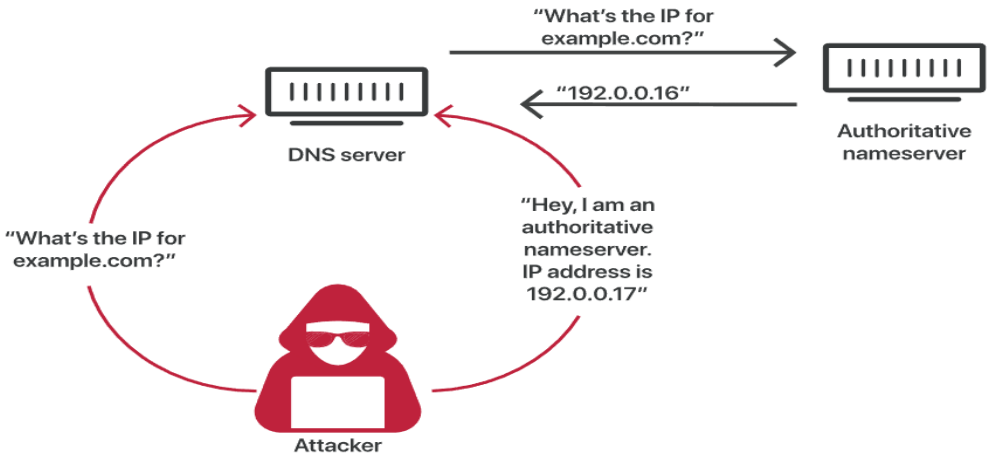

Figure 3 : DNS Cache Poisoning Process

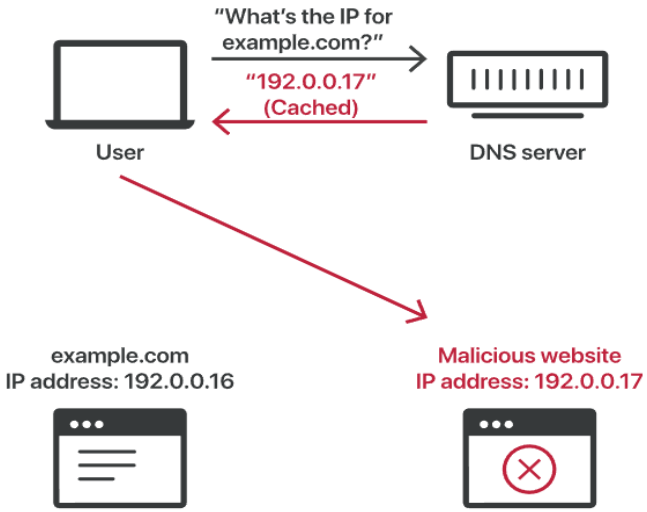

Figure 4 : Poisoned DNS cache

1.3 Problem Statement

Traditional DNS systems and monitoring tools are inherently flawed due to their inability to verify the authenticity of cached data. This vulnerability opens them up to spoofing and cache poisoning attacks. Current mitigation tactics typically involve manual inspection or rule-based detection methods, which are criticized for being slow, static, and challenging to maintain. Consequently, there is an urgent requirement for a real-time and automated system designed to effectively and accurately detect abnormal DNS behaviors, addressing these significant security concerns.

1.4 Objectives

The main objectives of this project are:

- To develop a **real-time DNS monitoring system** for detecting cache poisoning attempts.
- To implement **DNSSEC validation** to verify the authenticity of DNS responses.
- To use a **Random Forest machine learning model** for classifying suspicious DNS activity.
- To design a **Django-based web dashboard** for visualization and PDF report generation.
- To create a **lightweight, user-friendly tool** that enhances DNS-level security.

CHAPTER 2: SYSTEM DESIGN AND METHODOLOGY

The Domain Cache Detector is a sophisticated system that aims to identify DNS cache poisoning in real-time. It employs a machine learning model alongside DNSSEC (Domain Name System Security Extensions) validation to enhance its detection capabilities. The operation of this system is divided into four key stages, each playing a crucial role in the effective monitoring and analysis of DNS data to prevent potential security threats associated with cache poisoning.

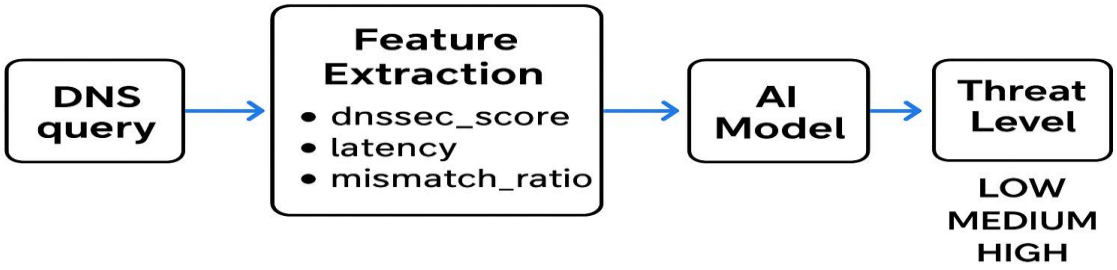

Figure 5 : Work Flow Diagram

2.1 DNS Query

The process initiates with the implementation of DNS queries for designated domains utilizing the dnspython library. This step involves gathering response information, which includes important metrics like Time to Live (TTL), IP addresses, and the duration of the query. The aggregated data from these responses serves as the foundational raw input for subsequent analytical processes.

2.2 Feature Extraction

From each DNS response, key features are extracted:

- **dnssec_score:** DNSSEC score quantifies the security of a DNS response through DNSSEC validation.
- **latency:** Latency refers to the response time, measured in milliseconds, of a DNS query.
- **mismatch_ratio:** Analyzes returned IPs against known IPs to detect suspicious deviations. These features are important indicators of possible anomalies or cache poisoning attempts

2.3 AI Model

The extracted features from DNS response data are processed by a **Random Forest machine learning model**, which analyzes them to determine the likelihood of a DNS cache poisoning attempt. The model evaluates multiple parameters, such as latency, TTL variation, mismatch ratio, and DNSSEC score, to identify patterns that distinguish normal DNS behavior from malicious activity. The resulting prediction generates a **threat score**, representing the probability of the DNS response being compromised. This score is then used to classify each response into different **threat levels**—Low, Medium, or High—reflecting the overall risk associated with that particular DNS interaction.

2.4 Threat Level

The system classifies each DNS response into a specific **threat level** based on the output of the machine learning model, helping administrators quickly assess the severity of potential risks. A **LOW** threat level indicates normal DNS behavior with no evidence of tampering, while a **MEDIUM** level suggests unusual or suspicious activity that might require closer observation. A **HIGH** threat level signals a strong possibility of DNS cache poisoning or spoofing, meaning the DNS data may have been altered or originated from an untrusted source. These classifications are presented on the **Django web dashboard**, where administrators can monitor real-time alerts, track threat patterns through visual charts, and generate **PDF reports** for further investigation or documentation. This layered threat-level system enhances situational awareness and supports prompt responses to DNS-related security incidents.

CHAPTER 3: LITERATURE REVIEW

The Domain Name System (DNS) has undergone several evolutions since its creation, yet its fundamental design remains vulnerable to manipulation. Researchers have studied various DNS-based attacks and proposed multiple methods for ensuring DNS integrity.

3.1 Historical Background

The DNS protocol was initially designed for scalability and usability, not for security. In the early Internet era, data transmission was based on mutual trust among nodes, with little consideration for adversarial threats. However, as the Internet grew, attackers began exploiting DNS weaknesses to redirect traffic, perform phishing attacks, and disrupt services.

3.2 DNS Cache Poisoning Research

- Kaminsky Attack (2008):

The Kaminsky vulnerability demonstrated how attackers could flood DNS resolvers with spoofed responses and successfully inject false records into caches. This incident highlighted the urgent need for DNSSEC and stronger validation mechanisms.

- Cryptographic Validation (DNSSEC):

DNSSEC introduced digital signatures and chain-of-trust verification. While effective in theory, global adoption remains below 20% due to deployment complexity and backward compatibility issues.

- Machine Learning Approaches:

Recent studies applied supervised and unsupervised learning algorithms to detect anomalies in DNS traffic. For instance, Random Forest, Support Vector Machines (SVM), and Neural Networks are used to identify malicious domains, fast-flux behaviour, and spoofed DNS replies.

Table 1 : Comparative Studies

Approach	Key Method	Limitation
Rule - based Detection	Uses predefined rules to identify anomalies	High false positives, not adaptive
Signature – based	Matches patterns of known attacks	Cannot detect zero-day threats
Statistical Models	Monitors traffic distribution	Needs large, clean datasets
ML – based Detection	Learns patterns from data, detects anomalies	Needs periodic retraining

CHAPTER 4: SYSTEM DESIGN

4.1 Architecture Overview

The architecture of the **Domain Cache Detector** is composed of four main components, each performing a specific function in the detection process:

1. **DNS Query Engine** – Responsible for sending DNS requests and collecting responses using the **dnspython** library, which provides raw data for further analysis.

2. **Feature Extractor** – Processes each DNS response to extract measurable parameters such as **TTL**, **latency**, and **mismatch ratio**, forming the dataset for the AI model.

3. **Machine Learning Analyzer** – Utilizes the **Random Forest algorithm** to evaluate the extracted features and classify DNS responses into different **threat levels** based on detected anomalies.

4. **Web Dashboard** – Developed using **Django**, this interface visually presents the detection results, real-time alerts, and analysis reports, allowing administrators to monitor and manage DNS security efficiently.

Table 2 : Technology Stack Overview

Layer	Technology
Front - End	HTML, CSS, JavaScript
Back - End	Django Framework
Machine Learning	Scikit – learn (Random Forest)
DNS Queries	dnspython
Visualization	Django Templates
Deployment	Localhost (127.0.0.1:8000)

4.2 Algorithms Used

Algorithm 1: Random Forest

- Input: DNS features (TTL, latency, mismatch ratio, dnssec_score)
- Output: Threat label (Low / Medium / High)

Steps:

1. Import dataset containing DNS response features.
2. Split dataset into training and testing sets.
3. Initialize multiple Decision Trees (n estimators).

Algorithm 2: DNSSEC Verification

1. Fetch DNS response for domain using dnspython.
2. Check DNSKEY and RRSIG records for authenticity.
3. Assign a “DNSSEC score” based on verification success or failure.

CHAPTER 5 IMPLEMENTATION DETAILS

5.1 “dnspython” Integration

The **dnspython** library is a crucial part of the Domain Cache Detector, as it is used to send DNS queries and collect important response data for further analysis. It allows the system to perform detailed lookups and gather features such as **response time (latency)**, **TTL (Time to Live)**, and **DNSSEC validation results**, which are later used by the machine learning model to identify potential DNS poisoning attempts.

When the system queries a domain, it doesn’t just retrieve the IP address — it also checks whether the response is **secure or fake**. This is done through **DNSSEC (Domain Name System Security Extensions)**, which adds special security records inside the DNS data to verify authenticity. These include:

- **DNSKEY:** Contains a public key used to verify digital signatures.
- **RRSIG:** The actual digital signature attached to the DNS record.
- **DS:** Connects one level of the DNS hierarchy to another securely, ensuring trust across domains.

5.2 Django Integration

Django acts as both the **controller** and **visualizer** in this system.

- **Backend logic:** Handles requests, responses, and triggers the ML module.
- **Frontend interface:** Displays graphs, status indicators, and generates PDF reports.

5.3 Machine Learning Model Training

A labeled dataset of DNS responses was created — some normal and some artificially poisoned using spoofed responses.Data pre - processing was done using pandas and NumPy, followed by feature scaling.

5.4 Dashboard Interface

The Django dashboard provides:

- Real-time monitoring view
- Threat level indicator
- Table of recent DNS responses
- PDF report download

5.5 Workflow Integration

The file ‘detector_worker.py’ performs backend operations and continuously feeds results to the dashboard.

CHAPTER 6: SECURITY AND PERFORMANCE ANALYSIS

6.1 Security Perspective

The **Domain Cache Detector** ensures strong DNS security by combining cryptographic verification and intelligent detection. **Authenticity** is maintained through DNSSEC, which verifies that DNS responses come from legitimate sources using digital signatures. **Integrity** is ensured by validating each response against a secure chain of trust, preventing any alteration or tampering of data. Additionally, the **machine learning model** detects abnormal DNS behavior even when DNSSEC validation is unavailable, analyzing features like latency, TTL variation, and mismatch ratio to identify possible cache poisoning attempts. This hybrid approach provides reliable, multi-layered protection against DNS spoofing and poisoning attacks.

Table 3 : Advantages over Traditional Methods

Feature	Traditional Tools	Domain Cache Detector
Real-time Detection	✗	✗
AI-Based Classification	✗	✓
DNSSEC Validation	Partial	Full
User Dashboard	✗	✓

CHAPTER 7: RESULTS / SCREENSHOTS

Figure 6 : Dashboard

Recent Events

TIME	DOMAIN	SCORE	LATENCY	MISMATCH	LEVEL	NOTES
2025-11-10 11:56:01	amazon.com	52	44.89	0.47	MEDIUM	Potential cache inconsistencies observed.
2025-11-10 11:55:54	dropbox.com	45	74.13	0.13	MEDIUM	Potential cache inconsistencies observed.
2025-11-10 11:55:47	cloudflare.com	81	159.5	0.16	LOW	No major issues detected. DNSSEC validation strong.
2025-11-10 11:55:38	banksecure.net	65	97.31	0.61	MEDIUM	Potential cache inconsistencies observed.
2025-11-10 11:55:29	adobe.com	49	156.76	0.83	MEDIUM	Potential cache inconsistencies observed.
2025-11-10 11:55:22	yahoo.com	70	48.73	0.97	MEDIUM	Potential cache inconsistencies observed.
2025-11-10 11:55:14	yahoo.com	42	22.87	0.71	MEDIUM	Potential cache inconsistencies observed.
2025-11-10 11:55:07	amazon.com	41	104.77	0	MEDIUM	Potential cache inconsistencies observed.
2025-11-10 11:54:58	stackoverflow.blog	67	53.31	0.27	MEDIUM	Potential cache inconsistencies observed.
2025-11-10 11:54:49	bing.com	70	72.82	0.57	LOW	No major issues detected. DNSSEC validation strong.
2025-11-10 11:54:40	bing.com	46	39.55	0.72	MEDIUM	Potential cache inconsistencies observed.
2025-11-10 11:54:31	adobe.com	57	118.68	0.4	MEDIUM	Potential cache inconsistencies observed.

Figure 7 : DNS Threat Monitoring Dashboard – Recent Events Table

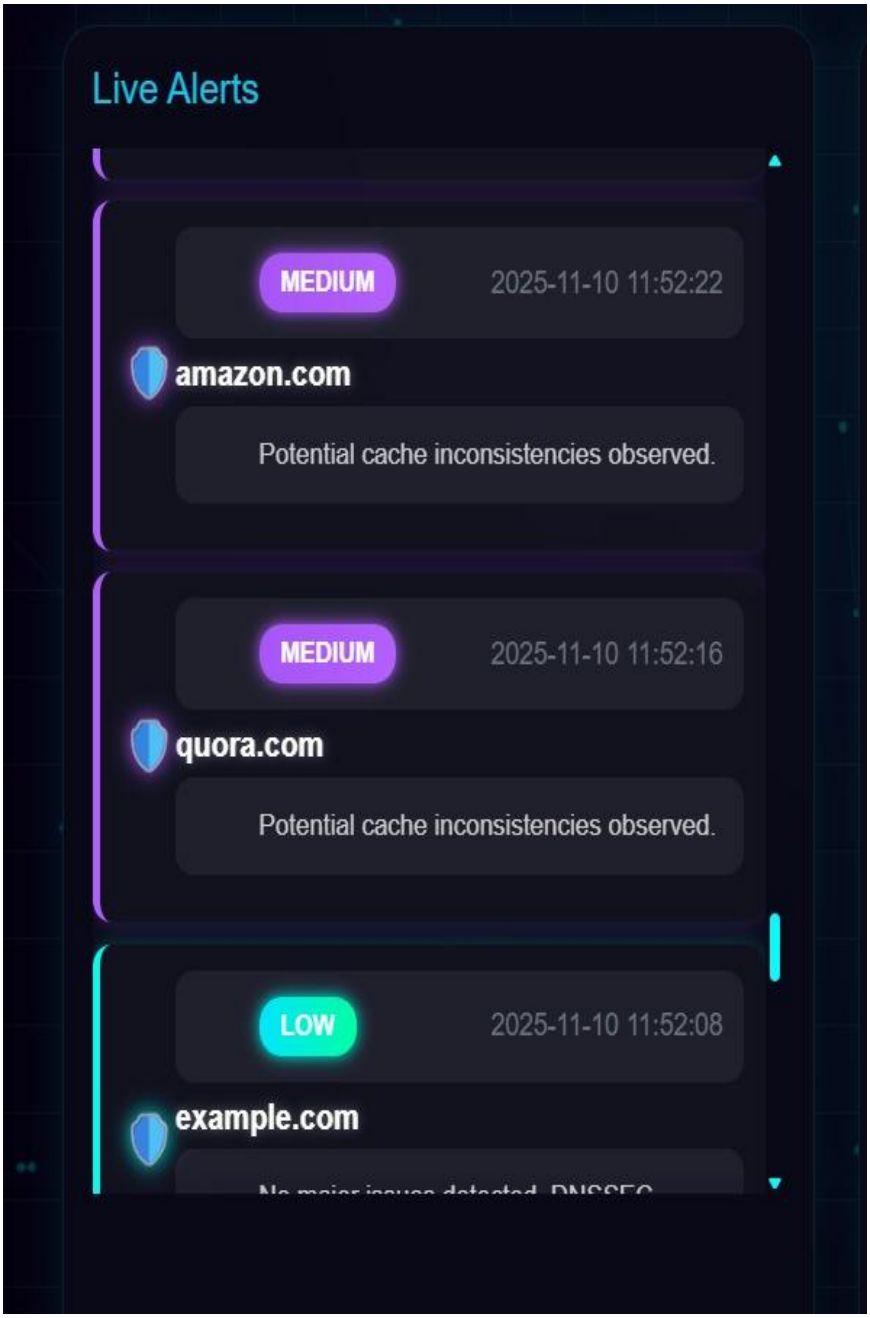

Figure 8 : Live DNS Threat Alerts Panel

Conclusion

The Domain Cache Detector project is designed to identify DNS cache poisoning attacks through a blend of DNSSEC validation and machine learning methodologies. It utilizes key DNS features, including DNSSEC score, latency, and mismatch ratio, and employs a Random Forest classifier to categorize the threat level as LOW, MEDIUM, or HIGH. The implementation is built using Python and the Django framework, resulting in a user-friendly dashboard that displays detection outcomes and allows for the generation of PDF reports. The system executes DNS queries on a pre-defined list of domains to analyze potential cache poisoning incidents.

While the project effectively showcases the potential of integrating AI with DNS security mechanisms, it is limited in scope; it operates on a fixed list of domains rather than engaging in continuous, real-time monitoring. This design choice was made to facilitate testing, minimize complexity, and ensure a controlled evaluation process during the development phase. Ultimately, the project confirms the viability of combining DNSSEC with machine learning techniques to enhance threat detection and awareness at the DNS level.

Future Works

While the current version provides an effective proof of concept, several enhancements can be implemented to make the system more powerful and production-ready:

1. **Real-Time Monitoring:** Enable continuous and automated domain monitoring instead of using a fixed list of domains. The system can dynamically fetch and analyze active DNS traffic in real time.

2. **Model Improvement:**
   Train the model with a larger dataset, include more DNS-related features (e.g., TTL variation, query type ratio), and explore advanced algorithms like XGBoost or Neural Networks.

3. **Live Alert System:**
   Implement email, SMS, or dashboard notifications to alert administrators immediately when a high-risk DNS response is detected.